SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0302	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	M Poojitha Reddy	**Reg. No.:**	192472352
Section / Batch:	2024	**Date:**	28-7-26

Code of Conduct

*I _**M Poojitha Reddy- 192472352**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **M Poojitha Reddy**

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Annexure A – Coding Challenge

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

1. Study and understand the assigned rule.
2. Explain the linguistic purpose of the rule.
3. Describe the conditions under which the rule should be applied.
4. Implement the rule in Python.
5. Prepare at least **10 test words** demonstrating:
    - Words that satisfy the rule.
    - Words that should remain unchanged.
6. Document the input, expected output, and actual output.
7. Explain any limitations or special cases encountered.

Phase 2: Team Integration

As a team, integrate all individual rule implementations into a complete Porter Stemmer.

The team must:

- Arrange the rules in the correct execution order.
- Resolve conflicts between rule implementations.
- Ensure that the output of one rule becomes the input to the next rule.
- Test the complete algorithm using at least **50 different English words**.
- Compare the team's results with the NLTK Porter Stemmer.
- Identify and explain any differences.

Phase 3: Reflection

Prepare a report addressing the following questions:

1. Why is the order of Porter Stemmer rules important?
2. What errors would occur if your assigned rule were skipped?
3. How did integrating multiple rules improve stemming accuracy?
4. Which rule was the most challenging to implement and why?
5. What improvements would you suggest for the Porter Stemming algorithm?

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach

Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1	20	25	20	15	15	95	95

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

STEP 2 : POTTER STEMMER

(“ZATION”-→IZE)

```
vowels = "aeiou"

def is_vowel(word, i):
    return word[i] in vowels

def measure(stem):
    m = 0
    prev_vowel = False

    for i in range(len(stem)):
        if is_vowel(stem, i):
            prev_vowel = True
        else:
            if prev_vowel:
                m += 1
            prev_vowel = False

    return m

def step2(word):
    if word.endswith("ization"):
        stem = word[:-7]
        if measure(stem) > 0:
            return stem + "ize"
    return word

word = input("Enter a word: ")

print("Original Word :", word)
print("After Step 2  :", step2(word))
```

ALGORITHM

1. The user enters a word.
2. It checks whether the word ends with "ization".
3. If "ization" is present, it removes the suffix.
4. The measure of the remaining stem is checked.
5. If the measure is greater than 0, "ize" is added.
6. The final stemmed word is displayed.

EXPLANATION

vowels = "aeiou"

This stores all the vowels in a string. It is used to check whether a character is a vowel or not.
def is_vowel(ch):
return ch in vowels
This function checks whether the given character is a vowel.
If the character exists in the vowels string, it returns True; otherwise, it returns False.
Example:
is_vowel('a') → True
is_vowel('b') → False
def measure(word):
This function calculates the measure value (m) of a word.
In Porter Stemmer, measure represents the number of vowel-consonant (VC) patterns present in a word.
It is used to decide whether a suffix removal rule can be applied.
m = 0
i = 0
n = len(word)
m stores the count of VC patterns.
i is used to traverse through the word.
n stores the length of the word.
while i < n and is_vowel(word[i]):
i += 1
This skips the starting vowels of the word.
The loop continues until a consonant is found.
while i < n:
while i < n and not is_vowel(word[i]):
i += 1
This skips all consonants in the word.
while i < n and is_vowel(word[i]):
i += 1
This skips all vowels after the consonant sequence.
if i < n:
m += 1
Whenever a vowel-consonant pattern is found, the measure value is increased by 1.
return m
Returns the final measure value of the word.
def step2(word):

This function implements Porter Stemmer Step 2.
In this program, only one Step 2 rule is implemented:
"ization" → "ize"
if word.endswith("ization"):
Checks whether the given word ends with the suffix "ization".
Examples:
organization → True
modernization → True
playing → False
stem = word[:-7]
Removes the last 7 characters ("ization") from the word.
Example:
organization
remove "ization"
Result:
organ
if measure(stem) > 0:
Calculates the measure value of the remaining stem.
The replacement rule is applied only if the measure is greater than 0.
return stem + "ize"
Replaces "ization" with "ize".
Example:
organization
organ + ize
organize
return word
If the word does not satisfy the rule, the original word is returned without any change.
word = input("Enter a word: ")
Takes a word from the user.
print("Original Word :", word)
Displays the input word.
print("Measure (m) :", measure(word))
Calculates and displays the measure value of the input word.
print("After Step 2 :", step2(word))
Applies Porter Stemmer Step 2 rule and displays the stemmed output.